Laboratory Assignment 4

Design of a Radix-4 Booth Multiplier

Shahan Antabian / Ayush Mohpatra
High-Level Digital Design

May 23, 2026



Contents

1 Objective **3**

2 Background: Booth Multiplication **3**
2.1 Radix-4 Booth 3

3 Hardware Implementation **4**
3.1 booth mult 4
3.2 Testbench 6

4 Simulation Results **10**

5 Resource Utilization **12**

6 Timing Analysis **12**

7 Conclusion **13**

1 Objective

The goal of this laboratory assignment is the design of a 16×16-bit signed multiplier. The multiplier must support an external control interface suitable for later encapsulation as an AXI4-Lite IP block managed by interrupts of the HPS in the Cyclone V SoC.

2 Background: Booth Multiplication

Binary multiplication of two operands of width m and n produces a result of width $m+n$. The classic shift-and-add method requires one partial-product addition for every 1 in the multiplier. The Booth algorithm reduces the number of additions whenever the multiplier contains runs of consecutive ones, by replacing a string $2^{j-1} + 2^{j-2} + \cdots + 2^{i}$ with the equivalent form $2^{j} - 2^{i}$.

2.1 Radix-4 Booth

The radix-4 variant inspects three bits per cycle, halving the number of iterations to $n/2$. With a 16-bit multiplier we therefore need only eight add-and-shift cycles. The corresponding action table is shown in Table 1.

Table 1: Radix-4 Booth action table.

m_{i+1}	m_i	m_{i-1}	Action
0	0	0	None
0	0	1	Add multiplicand
0	1	0	Add multiplicand
0	1	1	Add $2\times$ multiplicand
1	0	0	Subtract $2\times$ multiplicand
1	0	1	Subtract multiplicand
1	1	0	Subtract multiplicand
1	1	1	None

After each iteration the partial-product register is arithmetically shifted right by two positions so that the next three-bit window aligns with the encoder.

3 Hardware Implementation

3.1 booth mult

The complete description of the multiplier is given in Listing 1.

```
‘timescale 1 ns / 1 ps


module booth_mult (
   input  logic                clk,
   input  logic                resetn,
   input  logic                start,
   output logic                busy,
   output logic                irq,
   input  logic                ack,
   input  logic signed [15:0] data_a,
   input  logic signed [15:0] data_b,
   output logic signed [31:0] result
); //Interface definition: clk, resetn, start, busy, irq, ack, data_a, data_b, result


   // FSM definition: IDLE, RUN, DONE. Initiates the start of the multiplication,
      performs the 8 cycles of Booth’s algorithm.
   typedef enum logic [1:0] {IDLE = 2’b00, RUN = 2’b01, DONE = 2’b10} state_t;
   state_t state;


   logic signed [33:0] prod;  // 34-bit register: 17 bits for the upper half (
      accumulator) and 17 bits for the lower half (partial sum) product + encoder
      input)
   logic        [2:0]  count; // 3-bit counter for the 8 cycles of Booth’s algorithm
      (0 to 7)
   logic        start_d;  // for rising-edge detection on start


   // Signed multiplicand
   // Signed data_a:17 bits for Booth encoding. The MSB of data_a is copied to create
       the 17th bit, ensuring correct handling of negative values.
   wire signed [16:0] M = {data_a[15], data_a};




   logic signed [17:0] booth_out; // M or 2M or -M or -2M based on the last 3 bits of
       the product (Booth’s encoding output)


/* Booth’s encoding logic (combinational): based on the last 3 bits using the lookup
    table:
   000, 111: 0
   001, 010: +M
   011:      +2M
   100:      -2M
   101, 110: -M */




   always_comb begin
       case (prod[2:0])
           3’b000, 3’b111: booth_out = 18’sd0;
           3’b001, 3’b010: booth_out =  {M[16], M};   //  +M  (sign-ext 17 -> 18)
           3’b011:         booth_out =  {M, 1’b0};    //  +2M
```

```
            3'b100:          booth_out = -{M, 1'b0};    //  -2M
            3'b101, 3'b110: booth_out = -{M[16], M};   //  -M
            default:         booth_out = 18'sd0;
        endcase


        // Signed sum of accumulator (top 17 bits) and partial product
        upper = $signed(prod[33:17]) + booth_out;
    end


    // Synchronous reset and state transitions for the FSM.
    always_ff @(posedge clk) begin
        if (!resetn) begin
            prod    <= '0;
            count   <= '0;
            state   <= IDLE;
            irq     <= 1'b0;
            start_d <= 1'b0;
        end
        else begin
            start_d <= start;     // capture start for rising-edge detection


            case (state)


                //  IDLE: wait for start signal.
                IDLE: begin
                    if (start && !start_d) begin
                        prod  <= {17'b0, data_b, 1'b0};   // Initialize product: upper
                             half = 0, lower half = data_b
                        count <= 3'd0; // reset cycle count
                        state <= RUN;
                    end
                end


                // RUN: FSM remains in this state for 8 cycles, performing the Booth's
                     algorithm steps.
                RUN: begin
                    // Update the product by shifting right 2 bits for radix-4 Booth's
                         algorithm.
                    prod  <= $signed({upper, prod[16:0]}) >>> 2;
                    count <= count + 3'd1;


                    if (count == 3'd7) begin
                        state <= DONE;
                        irq   <= 1'b1;        // raise irq when multiplication is done
                    end
                end


                // DONE: wait for ack signal to clear irq and return to IDLE.
                DONE: begin
                    if (ack) begin
                        irq   <= 1'b0;
                        state <= IDLE;
                    end
                end


                default: state <= IDLE;
```

```
            endcase
        end
    end


    // busy signal is high when the FSM is in RUN or DONE states
    assign busy   = (state != IDLE);
    assign result = prod[32:1]; // Final result is the upper 32 bits of the product


endmodule
```

Listing 1: Radix-4 Booth multiplier (`booth_mult.sv`).

3.2 Testbench

```
`timescale 1 ns / 1 ps


module tb_booth;


   // interface signals for the radix-4 booth multiplier


   logic clk, resetn, start, ack;
   logic signed [15:0] data_a, data_b;
   logic signed [31:0] result;
   logic busy, irq;


   // instantiation of the booth multiplier module


   booth booth_instance(
       .clk(clk),
       .resetn(resetn),
       .start(start),
       .busy(busy),
       .irq(irq),
       .ack(ack),
       .data_a(data_a),
       .data_b(data_b),
       .result(result)
   );


   // clock generation: 20 ns period (50 MHz)


   always
       #10 clk = ~clk;


   /* all test stimulus:
      1- negative * positive
      2- negative * negative
      3- positive * positive
      4- positive * negative
   */


   initial
   begin
       // initial values for signals


       clk = 0;
       resetn = 1;
       start = 0;
       ack = 0;


       data_a = 0;
       data_b = 0;


       #5  resetn = 0;      // reset the system at time 5 ns
       #40 resetn = 1;      // release reset at time 45 ns
```

```
64
65
66          // apply first set of inputs at 30 ps (0.03 ns)
67          #30
68          data_a = -60;
69          data_b = 5;
70
71
72          #20 start = 1;        // start multiplication at time 95 ns
73          #60 start = 0;       // stop start signal at time 155 ns
74
75
76          // wait for irq signal to go high, indicating multiplication is done
77          wait (irq == 1);
78
79
80          #10 ack = 1;          // acknowledge to clear irq
81          #40 ack = 0;
82
83
84          #5  resetn = 0;
85          #40 resetn = 1;
86
87
88          // apply second set of inputs
89          #30
90          data_a = -60;
91          data_b = -10;
92
93
94          #20 start = 1;        // start multiplication
95          #60 start = 0;
96
97
98          // wait for result
99          wait (irq == 1);
100
101
102         #10 ack = 1;          // acknowledge
103         #40 ack = 0;
104
105
106         #5  resetn = 0;       // reset for the system
107         #40 resetn = 1;
108
109
110         // apply third set of inputs
111         #30
112         data_a = 60;
113         data_b = 15;
114
115
116         #20 start = 1;        // start multiplication
117         #60 start = 0;
118
119
120         // wait for irq signal to go high, indicating multiplication is done
121         wait (irq == 1);
122
123
124         #10 ack = 1;          // acknowledge
125         #40 ack = 0;
126
127
```

```
128         #5  resetn = 0;       // reset for the system
129         #40 resetn = 1;
130
131
132         // apply fourth set of inputs
133         #30
134         data_a = 200;
135         data_b = -8;
136
137
138         #20 start = 1;        // start multiplication
139         #60 start = 0;
140
141
142         // wait for result
143         wait (irq == 1);
144
145
146         #10 ack = 1;          // acknowledge
147         #40 ack = 0;
148
149
150         #20 $stop;
151     end
152
153
154     // monitoring of results
155
156
157     initial
158         $monitor($time,,,"clk=%b resetn=%b start=%b busy=%b irq=%b ack=%b A=%0d B=%0d
                result=%0d",
159                  clk, resetn, start, busy, irq, ack, data_a, data_b, result);
160
161
162 endmodule
```

Listing 2: Self-checking testbench (`tb_booth_mult.sv`).

4 Simulation Results

Table 2 lists the inputs for $data_a$ and $data_b$ used for booth multiplication with every possible signed integers combination used.

Table 2: Simulation results.

a	b	result	status
-60	5	-300	PASS
-60	-10	600	PASS
60	15	900	PASS
200	-8	-1600	PASS

Figure 1 shows a representative QuestaSim waveform for the multiplication $-60 \times 5 = -300$. The `busy` output rises on the cycle after `start`, remains high for the eight RUN iterations, and `irq` is asserted when `count` reaches 7. The HPS-side `ack` pulse returns the FSM to the IDLE state. Similarly, for other test cases we see that once the `ack` is set to high and the FSM is set to IDLE state we begin the next set of multiplications.

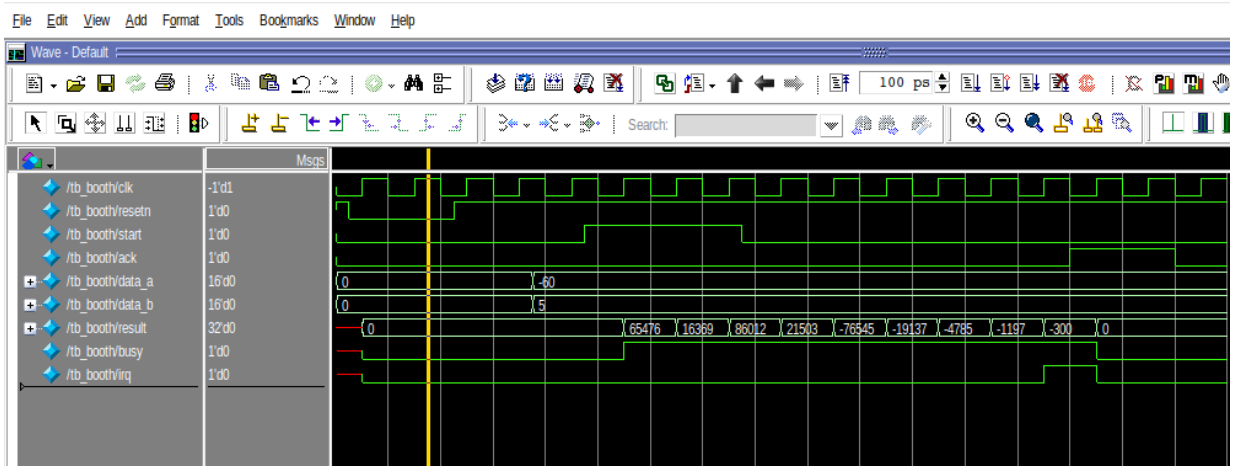


Figure 1: Multiplication of a negative multiplicand with a positive multiplier

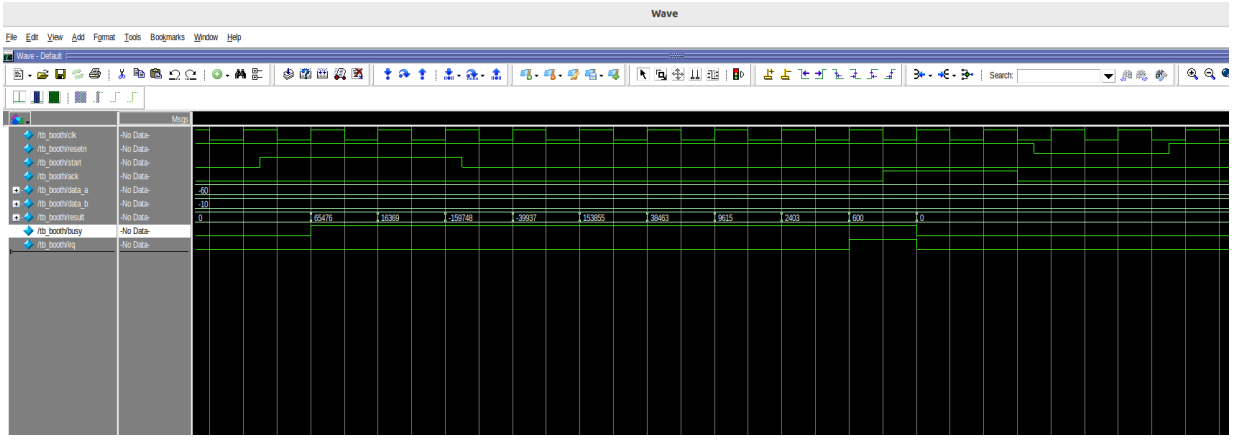


Figure 2: Multiplication of a negative multiplicand with a negative multiplier

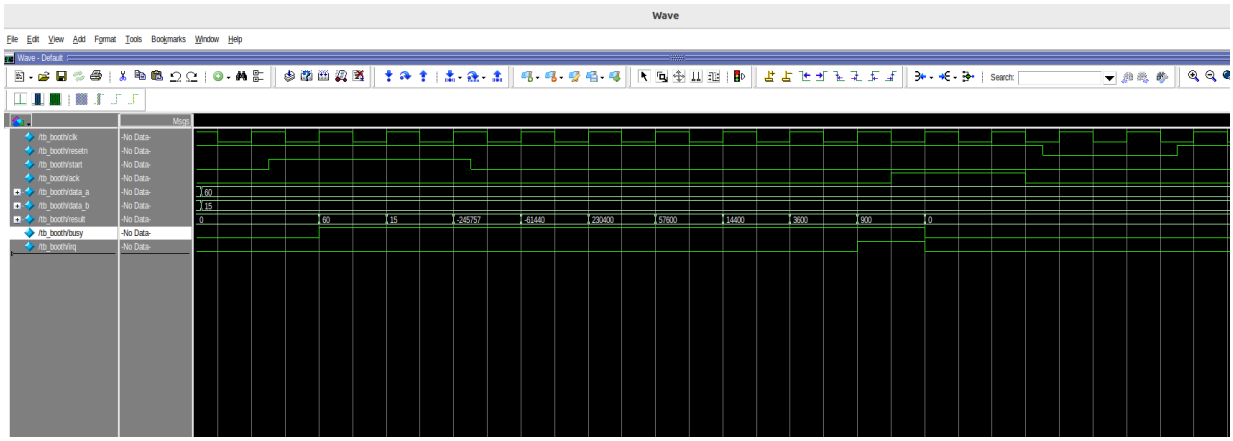


Figure 3: Multiplication of a positive multiplicand with a positive multiplier

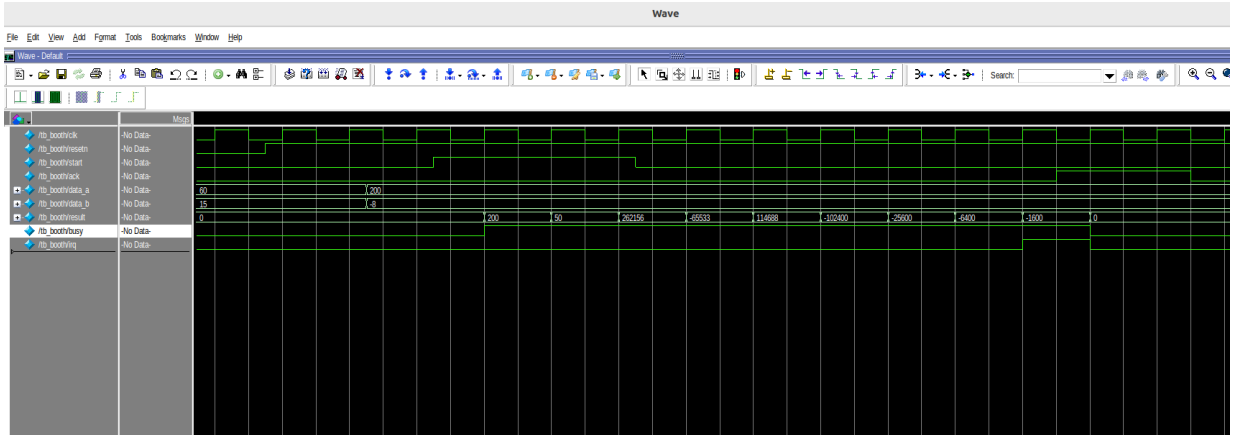


Figure 4: Multiplication of a positive multiplicand with a negative multiplier

5 Resource Utilization

The following table presents the FPGA resource utilization summary for the Radix-4 Booth Multiplier implementation synthesized using Quartus Prime.

Resource	Utilization
Logic utilization (in ALMs)	48 / 41,910 (<1%)
Total registers	38
Total pins	70 / 499 (14%)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0%)
Total DSP Blocks	0 / 112 (0%)
Total HSSI RX PCSs	0 / 9 (0%)
Total HSSI PMA RX Deserializers	0 / 9 (0%)
Total HSSI TX PCSs	0 / 9 (0%)
Total HSSI PMA TX Serializers	0 / 9 (0%)
Total PLLs	0 / 15 (0%)
Total DLLs	0 / 4 (0%)

Table 3: Quartus Prime Resource Utilization Summary

Target Device: Cyclone V (5CSXFC6D6F31C6)

Top-Level Entity: booth_mult

Compilation Status: Successful

The analysis and synthesis reports indicate that the booth multiplier IP is implemented using 38 registers.

---

6 Timing Analysis

The timing analysis was performed using the Quartus Prime Timing Analyzer for different operating conditions including slow and fast timing models at varying temperature ranges. The table below summarizes the setup, hold, and minimum pulse width timing results for the clock signal `clk`.

The timing analysis indicates negative setup slack values under both slow and fast timing models, suggesting timing violations in the current design configuration. However, hold timing requirements are satisfied in all cases with positive slack values and zero Total Negative Slack (TNS).

Timing Model	Analysis Type	Slack (ns)	TNS (ns)
Slow 1100mV 85C	Setup	-2.236	-52.662
Slow 1100mV 85C	Hold	0.237	0.000
Slow 1100mV 85C	Minimum Pulse Width	-0.394	-20.300
Slow 1100mV 0C	Setup	-2.379	-52.248
Slow 1100mV 0C	Hold	0.225	0.000
Slow 1100mV 0C	Minimum Pulse Width	-0.394	-21.369
Fast 1100mV 85C	Setup	-0.841	-14.651
Fast 1100mV 85C	Hold	0.140	0.000
Fast 1100mV 85C	Minimum Pulse Width	-0.085	-3.131
Fast 1100mV 0C	Setup	-0.757	-11.194
Fast 1100mV 0C	Hold	0.127	0.000
Fast 1100mV 0C	Minimum Pulse Width	-0.084	-3.134

Table 4: Timing Analyzer Summary for Clock Signal `clk`

7 Conclusion

A 16-bit x 16-bit signed radix-4 Booth multiplier has been described in SystemVerilog and verified through functional simulation using QuestaSim. The design completes a multiplication in eight clock cycles and uses the signals(`start` / `busy` / `irq` / `ack`) that allows direct encapsulation as an AXI4-Lite IP in the subsequent laboratory assignment. The testbench confirms correct operation across all sign combinations and edge cases.